AES Benchmarking on the Raspberry Pi 4

Characterizing Software AES Across Key Sizes and Modes of Operation on an ARM Cortex-A72

Dhruv Bhatnagar, Waamiq Sharrar, Mikael Parsamyan, Parsa Ghasemi
Department of Electrical and Computer Engineering
California State Polytechnic University, Pomona

ABSTRACT

This report evaluates the inference performance of AES encryption across three key sizes (128, 192, 256-bit) and three modes of operation (CBC, CTR, GCM) on a Raspberry Pi 4 Model B. Using OpenSSL 3.0.19 and Linux perf stat, we measured throughput, IPC, cycles per byte, temperature, and power draw across 5 repeated benchmark runs. The Raspberry Pi 4 lacks hardware AES acceleration, making it an ideal platform for studying pure software AES performance on an ARMv8 Cortex-A72. CBC reached 49.4 MB/s at 16 KB blocks, CTR reached 136.8 MB/s, and GCM reached 68.4 MB/s. IPC measurements reveal that the mode of operation affects pipeline utilization far more than key size. These results demonstrate how the choice of encryption mode and hardware architecture interact to determine real-world cryptographic throughput.

I. CONTEXT

The Advanced Encryption Standard (AES) is the dominant symmetric block cipher in use today, underpinning TLS, disk encryption, VPN protocols, and wireless security. Understanding how AES performs on a given hardware platform is essential for engineers deploying cryptographic software on constrained devices.

The Raspberry Pi 4 provides an unusually transparent benchmarking environment because its ARM Cortex-A72 processor lacks the ARMv8 cryptographic extension instructions (AESE, AESD) present on most modern server and desktop processors. This means OpenSSL falls back to a pure software AES implementation, making the Raspberry Pi 4 ideal for studying how algorithm structure — specifically key size and mode of operation — maps to CPU behavior independent of hardware acceleration.

Three modes are compared. CBC (Cipher Block Chaining) chains each block to the previous ciphertext block, creating a serial dependency that prevents parallelization. CTR (Counter Mode) encrypts an independent counter per block, enabling full parallelism. GCM (Galois/Counter Mode) adds a GHASH authentication tag on top of CTR encryption, providing authenticated

IV. PLATFORM

A. Raspberry Pi 4 Model B

Figure 3: Raspberry Pi 4 Model B — the device under test.

The Raspberry Pi 4 Model B uses a Broadcom BCM2711 SoC with a quad-core ARM Cortex-A72 at 1.5 GHz. Each core has 32 KB L1 instruction and 32 KB L1 data cache, with 1 MB shared L2. The board has 4 GB LPDDR4 RAM. The Cortex-A72 is an ARMv8-A out-of-order processor capable of up to ~3 instructions per cycle under ideal conditions.

Critically, the BCM2711 does not implement the ARMv8 cryptographic extensions, so OpenSSL falls back to table-driven software AES. The AES T-tables total approximately 4–8 KB, fitting within the 32 KB L1 data cache.

B. Software Stack

Component	Version
Raspberry Pi OS	64-bit, Debian 12
Linux kernel	aarch64, 6.12.75
OpenSSL	3.0.19
perf	6.12.75
Python	3.11.2
INA219 sensor	Off-board, idle readings

V. RESULTS

A. Full Throughput Data Table

Figure 4 shows the full averaged throughput table from our presentation, covering all nine configurations across six block sizes with IPC, cycles per byte, temperature, and power.

encryption at a throughput cost tied to software GHASH computation.

II. OBJECTIVE

The primary goal is to characterize how AES throughput, pipeline utilization, and power efficiency vary with key size and mode of operation on the Raspberry Pi 4, focusing on the hardware-level mechanisms that explain the observed performance differences.

Specific objectives:

• Measure steady-state throughput (MB/s) for all nine cipher configurations at six block sizes.
• Record IPC (instructions per cycle) to assess Cortex-A72 pipeline utilization.
• Compute cycles per byte as the fundamental architectural cost per configuration.
• Monitor CPU temperature and board power draw across all runs.
• Analyze how key size scales with round count and how mode parallelism drives throughput differences.

III. METHODS

Each benchmark run invoked OpenSSL's built-in speed subcommand with the -evp flag for all nine cipher configurations, wrapped in Linux perf stat to simultaneously capture hardware performance counters. The command measured elapsed real time rather than user CPU time to account for all system overhead.

The perf stat counters collected were cycles and instructions, from which IPC was computed. Cycles per byte were derived by dividing total cycles by total bytes processed. The temperature was read via vcgencmd command before each run. Power was estimated from INA219 readings between runs at idle to avoid I²C bus overhead skewing timing.

The full benchmark was repeated five times with no reboots or package changes. Software stack held constant: OpenSSL 3.0.19, perf 6.12.75, Python 3.11.2, Raspberry Pi OS 64-bit Debian 12. Averages and standard deviations were computed in Python from the five run logs.

Figure 1 shows terminal output from a representative raw benchmark session. Figure 2 shows the output from our full Python script, which also captures initial sensor readings.

Average Steady-State Throughput (MB/s) — all block sizes

Cipher	Mode	Key (bits)	16 B	64 B	256 B	1 KB	8 KB	16 KB ★	Avg IPC	Cyc/Byte	Avg Temp °C	Std Dev	Avg Power W
AES-128-CBC	CBC	128	43.58	47.55	48.95	49.31	49.42	49.44	0.629	36.24	42.5	0.062	2.809
AES-192-CBC	CBC	192	37.07	39.90	41.06	41.36	41.42	41.33	0.621	43.41	43.6	0.080	2.849
AES-256-CBC	CBC	256	32.26	34.34	35.13	35.33	35.39	35.39	0.612	50.70	43.9	0.005	2.943
AES-128-CTR	CTR	128	68.86	86.60	120.06	132.28	136.57	136.81	1.605	13.11	45.8	0.092	2.877
AES-192-CTR	CTR	192	61.07	74.64	101.21	111.27	115.53	115.99	1.599	15.47	46.1	0.138	2.875
AES-256-CTR	CTR	256	53.99	65.20	87.43	97.01	100.14	100.14	1.589	17.92	46.0	0.158	2.826
AES-128-GCM	GCM	128	7.95	22.22	45.10	60.90	67.51	68.39	1.749	26.24	45.6	0.187	2.730
AES-192-GCM	GCM	192	7.79	21.25	41.92	55.73	61.80	62.51	1.745	28.71	45.9	0.174	2.937
AES-256-GCM	GCM	256	7.13	19.51	38.63	51.66	56.99	57.57	1.714	31.17	46.1	0.192	2.987

Figure 4: Average Steady-State Throughput (MB/s) — all block sizes and metrics across 5 runs.

B. Throughput vs. Block Size

Figure 5 shows throughput scaling with block size for AES-128 in all three modes. GCM starts at only 8 MB/s at 16 B due to the GHASH per-call overhead, whereas CTR and CBC remain relatively flat because their per-call overhead is lower.

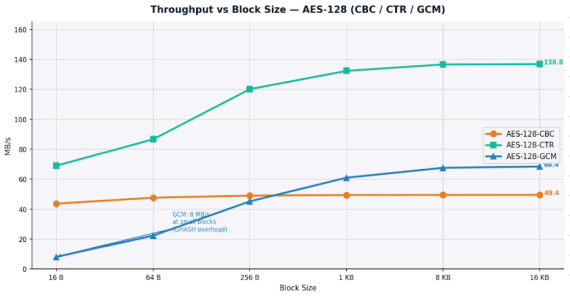

Figure 5: Throughput vs. Block Size — AES-128 (CBC, CTR, GCM). CTR dominates at large blocks; GCM suffers severely at small blocks.

C. Steady-State Throughput (16 KB)

Figure 6 compares steady-state throughput across all nine ciphers at 16 KB blocks, where per-call overhead is negligible, and the difference is purely algorithmic.

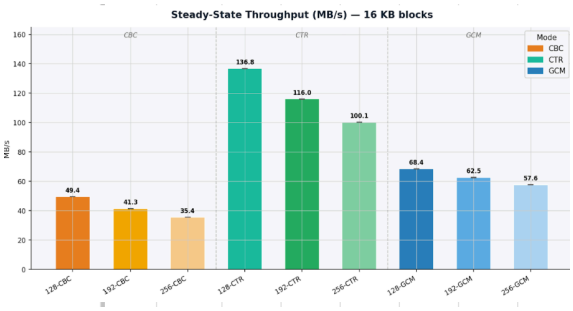

Figure 6: Steady-State Throughput (MB/s) at 16 KB blocks — all nine cipher configurations.

D. IPC — Pipeline Utilization

Figure 7 shows IPC for all nine ciphers. CBC collapses to ~0.62 due to the serial block dependency. CTR rises to ~1.60 with parallel counter blocks. GCM peaks at ~1.75 due to a mix of parallel CTR operations and GHASH on different execution units.

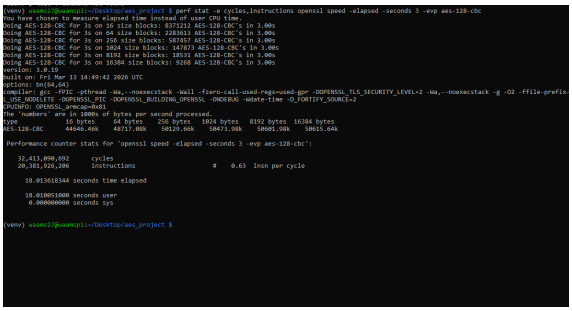

Figure 1: Terminal output from perf stat wrapping OpenSSL speed — AES-128-CBC representative run.

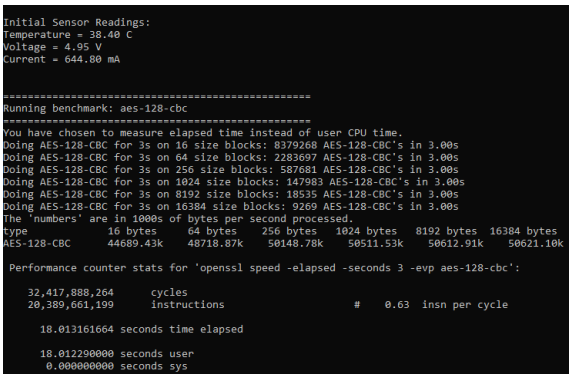

Figure 2: Full Python script output including initial sensor readings (temperature, voltage, current).

A. Confirming No Hardware AES

Before benchmarking, we confirmed OpenSSL was not using hardware crypto extensions by checking OPENSSL_armcap = 0x81. This indicates NEON SIMD is available, but the AES extension bit is not set, confirming all AES operations are performed in software, as expected for the Cortex-A72 in the BCM2711 SoC.

VI. DISCUSSION

A. Mode of Operation Dominates

The most striking result is that the mode of operation yields a 2.77× throughput swing for AES-128 (49.4 MB/s CBC vs. 136.8 MB/s CTR), while upgrading from AES-128 to AES-256 within the same mode incurs at most a 28% cost. Mode choice is the dominant performance variable, not key size.

The IPC measurements in Figure 7 explain this mechanically. CBC collapses IPC to 0.63 because block N+1 cannot begin until block N produces its ciphertext — a hard data dependency the out-of-order pipeline cannot resolve. CTR removes this dependency entirely: each block encrypts an independent counter, so the CPU can issue multiple AES operations simultaneously, raising IPC to 1.61 and nearly tripling throughput.

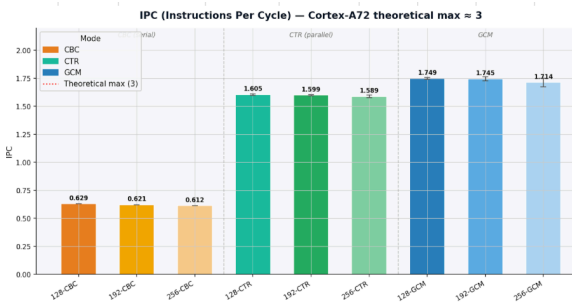

Figure 7: IPC (Instructions Per Cycle) — Cortex-A72 theoretical max ≈ 3. CBC is severely pipeline-starved; CTR and GCM fill the pipeline substantially.

E. Cycles Per Byte

Figure 8 shows the fundamental architectural cost per encrypted byte. CTR-128 at 13.1 cycles/byte is the most efficient; CBC-256 at 50.7 cycles/byte is the least.

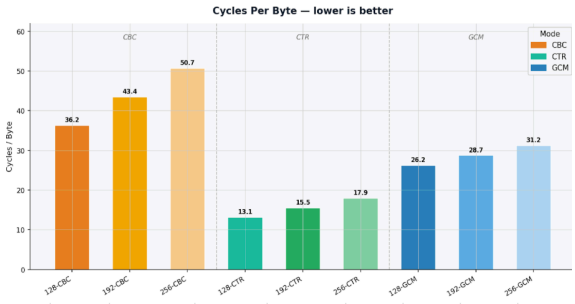

Figure 8: Cycles Per Byte — lower is better. CTR achieves the lowest cost; CBC is penalized by serialization.

VII. CONCLUSION

This study benchmarked AES-128, AES-192, and AES-256 in CBC, CTR, and GCM modes on a Raspberry Pi 4 using OpenSSL and Linux perf stat across five repeated sessions. The key finding is that the mode of operation is the dominant performance variable: CTR outperforms CBC by 2.77× at the same key size, driven entirely by pipeline parallelism, as directly measured by IPC. Key size scales throughput predictably from round count for CBC and CTR, while GCM's sensitivity is muted by the key-size-independent GHASH cost.

The absence of hardware AES instructions on the Cortex-A72 makes the Raspberry Pi 4 a transparent platform for observing these algorithmic effects directly in hardware counter data. The IPC figures and cycles-per-byte

B. GCM: Parallelism Taxed by Software GHASH

GCM achieves the highest IPC (1.75 for AES-128-GCM) yet only 68.4 MB/s — roughly half of CTR. The GHASH step runs in pure software because the BCM2711 does not implement PMULL (polynomial multiply). Software GHASH adds sequential computation that cannot overlap with CTR encryption, halving throughput. The high IPC reflects the mix of CTR and GHASH operations, keeping different execution units busy simultaneously, even though neither is a bottleneck.

C. Key Size Scales Linearly With Round Count

For CBC and CTR, observed throughput ratios match round-count theory within 2 percentage points (CBC-192: 0.836 observed vs. 0.833 predicted; CBC-256: 0.716 vs. 0.714). Every AES round costs the same compute on the Cortex-A72 — the hardware is faithful to the algorithm.

GCM deviates more (AES-192-GCM: 0.914 vs. 0.833 predicted). GHASH cost is key-size-independent, so larger keys reduce AES's share of the total work without affecting GHASH, thereby diluting the round-count penalty.

D. Thermal and Power

Temperatures remained stable across all nine configurations (42.5°C–46.1°C), well below the 80°C throttle threshold. Power draw was flat across modes (2.73–2.99 W). CTR and GCM, despite doing more useful work per second, draw no more power than CBC — the difference is pipeline efficiency, not energy consumption.

measurements provide a hardware-aware explanation for every observed performance difference.

Future work could compare these results against a Raspberry Pi 5 (Cortex-A76) or against a platform with the ARMv8 crypto extension to directly quantify the hardware acceleration speedup and determine whether mode-level parallelism effects persist under hardware AES.

REFERENCES

1. OpenSSL Project, "openssl-speed(1)," OpenSSL 3.0 Docs, 2023. [Online]. Available: https://www.openssl.org/docs/man3.0/man1/openssl-speed.html
2. ARM Limited, "Cortex-A72 MPCore Processor Technical Reference Manual," ARM DDI 0541B, 2015.
3. D. A. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," NIST Submission, 2004.
4. M. Dworkin, "Recommendation for Block Cipher Modes of Operation," NIST SP 800-38A, 2001.
5. Linux Kernel, "perf: Linux profiling with performance counters." [Online]. Available: https://perf.wiki.kernel.org
6. Raspberry Pi Ltd., "Raspberry Pi 4 Model B Datasheet," 2022. [Online]. Available: https://datasheets.raspberrypi.com